
Contents

Logica digitale	1
Circuiti combinatori e sequenziali	1
Circuiti combinatori	2
Decoder	2
Multiplexer	3
Addizionatori(full-adder)	5
ALU RISC-V ad 1 bit	5
Sottrazione	7
ALU RISC-V a 32 bit	7
NOR	9
Set on Less Than(SLT)	9
Overflow detection	13
Branch if Equal (beq)	13
Relazionare Opcode ed ingressi	14
Leggi delle algebre di Boole	16
Forme canoniche	17
Formula normale disgiuntiva	17
Formula normale congiuntiva	17
Formula di maggioranza	17
Dalla tavola di verità alla funzione logica	17

Logica digitale

Gli elementi base con cui sono costruiti i calcolatori sono detti **circuiti digitali**, sono dispositivi in grado di elaborare valori digitali(0, 1)

Un circuito digitale trasforma segnali digitali di ingresso in segnali digitali in uscita.

Gli elementi primitivi di questi circuiti sono dette **porte logiche**, esse sono create a partire dai transistor cioè un interruttore binario(se viene messo in ingresso un valore alto, restituisce un valore basso e viceversa). I transistor combinati insieme creano le porte logiche.

Circuiti combinatori e sequenziali

I circuiti sono detti combinatori quando l'uscita è una funzione dell'ingresso. I circuiti sono detti sequenziali se l'uscita è funzione oltre che dell'ingresso anche di uno stato(un circuito che ha memo-

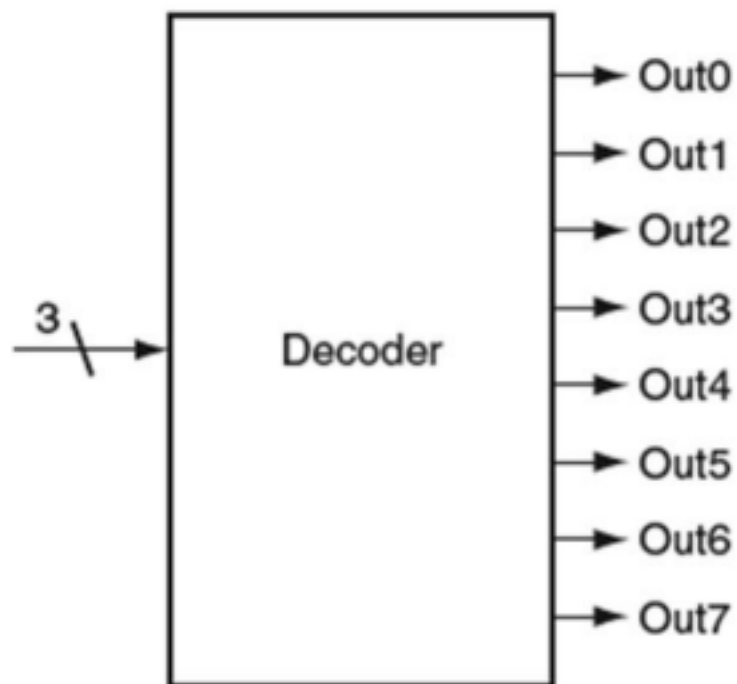
ria).

I circuiti combinatori uniscono più porte logiche per creare funzioni più complesse (convenzione: se due linee si incrociano, non c'è connessione a meno del puntino).

Circuiti combinatori

Decoder

Un decoder prende un numero n di bit in ingresso e lo usa per selezionare le 2^n linee in uscita.



a. A 3-bit decoder

Figure 1: alt text

Questo componente può tornarci utile per convertire numeri binari in numeri decimali.

Multiplexer

Un multiplexer è un circuito con 2 ingressi, 1 uscita e un ingresso di controllo. La linea di controllo determina quale dei due ingressi deve essere inviato all'uscita.

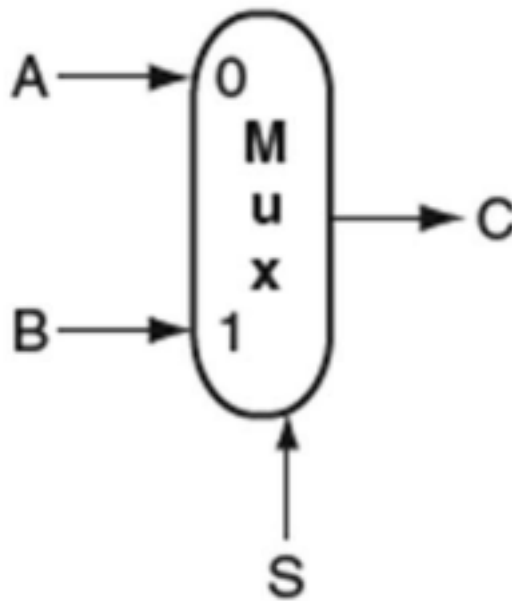
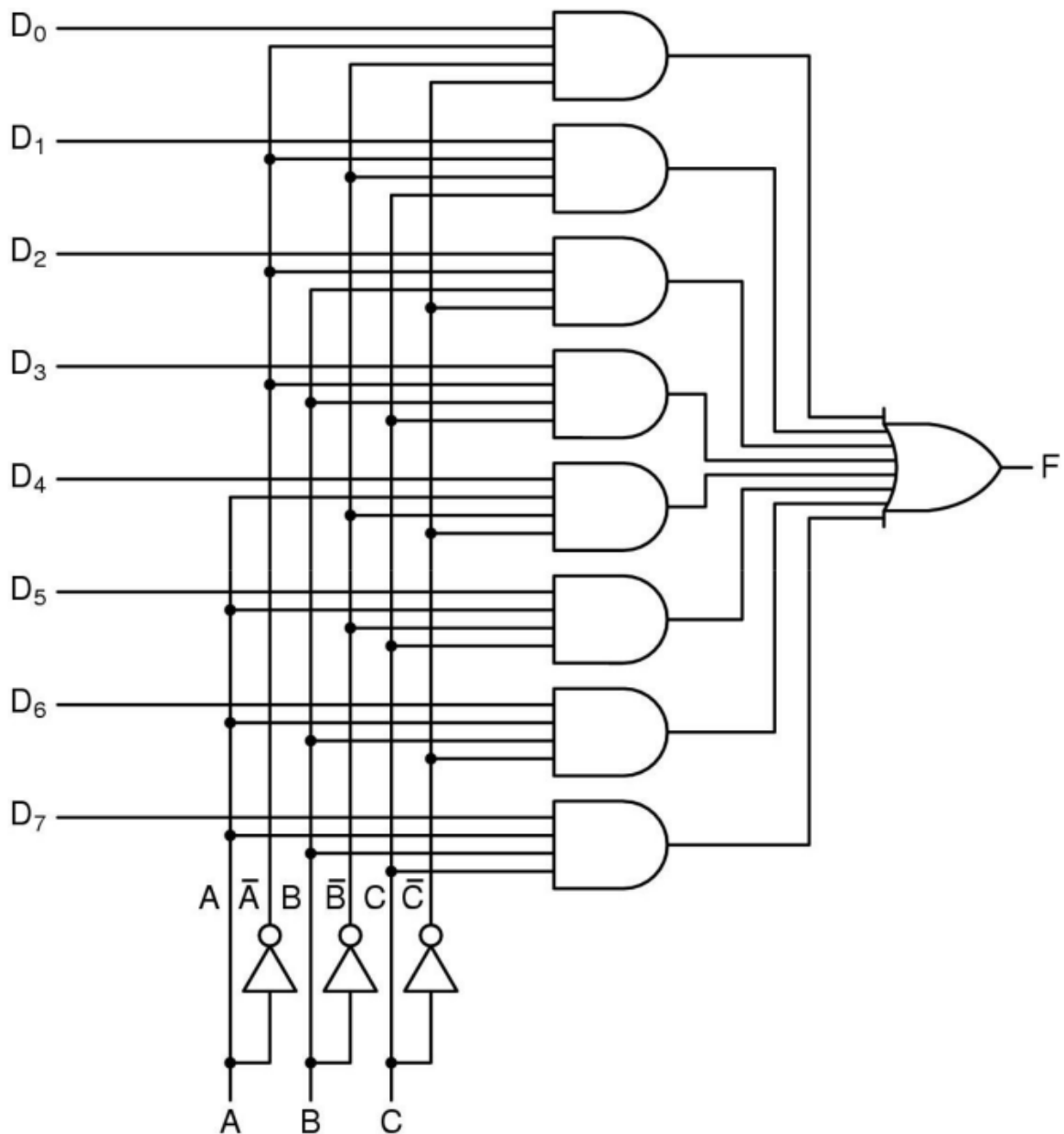


Figure 2: alt text

In generale possiamo avere 2^n in ingressi, un uscita e n ingressi di controllo.

Un multiplexer è un decoder dove ogni AND raccoglie un ingresso e un or finale.



↴

Figure 3: alt text

Un multiplexer può anche essere realizzato con 2 ingressi che però raccolgono 32 bit per ingresso e che fanno uscire 32 bit. L'ingresso di controllo resta sempre a 1 bit.

Questi due tipi di multiplexer sono solitamente uniti per creare funzioni più complesse.

Addizionatori(full-adder)

Gli addizionatori sommano due numeri binari, sono composti da:

- a e b, i due numeri da sommare
- CarryIn, rappresenta il riporto in ingresso che va sommato ad a+b
- Sum che restituisce il risultato a+b
- CarryOut, rappresenta il riporto in uscita

Questo addizionatore è detto full-adder perchè considera anche il riporto.

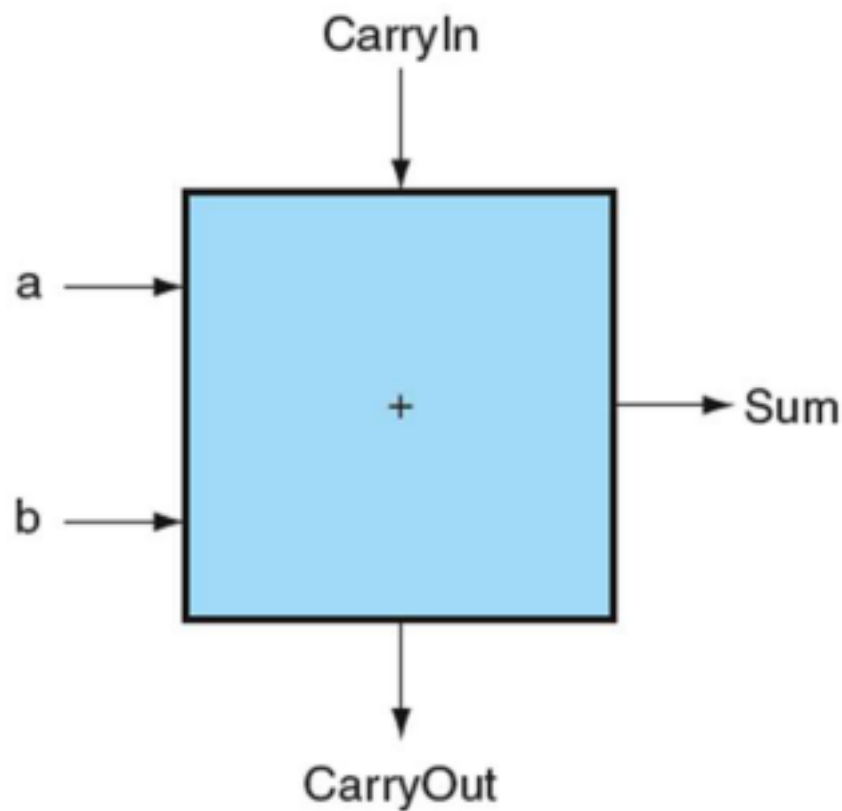


Figure 4: alt text

ALU RISC-V ad 1 bit

L'ALU in RISC-V è fatta come un circuito combinatorio:

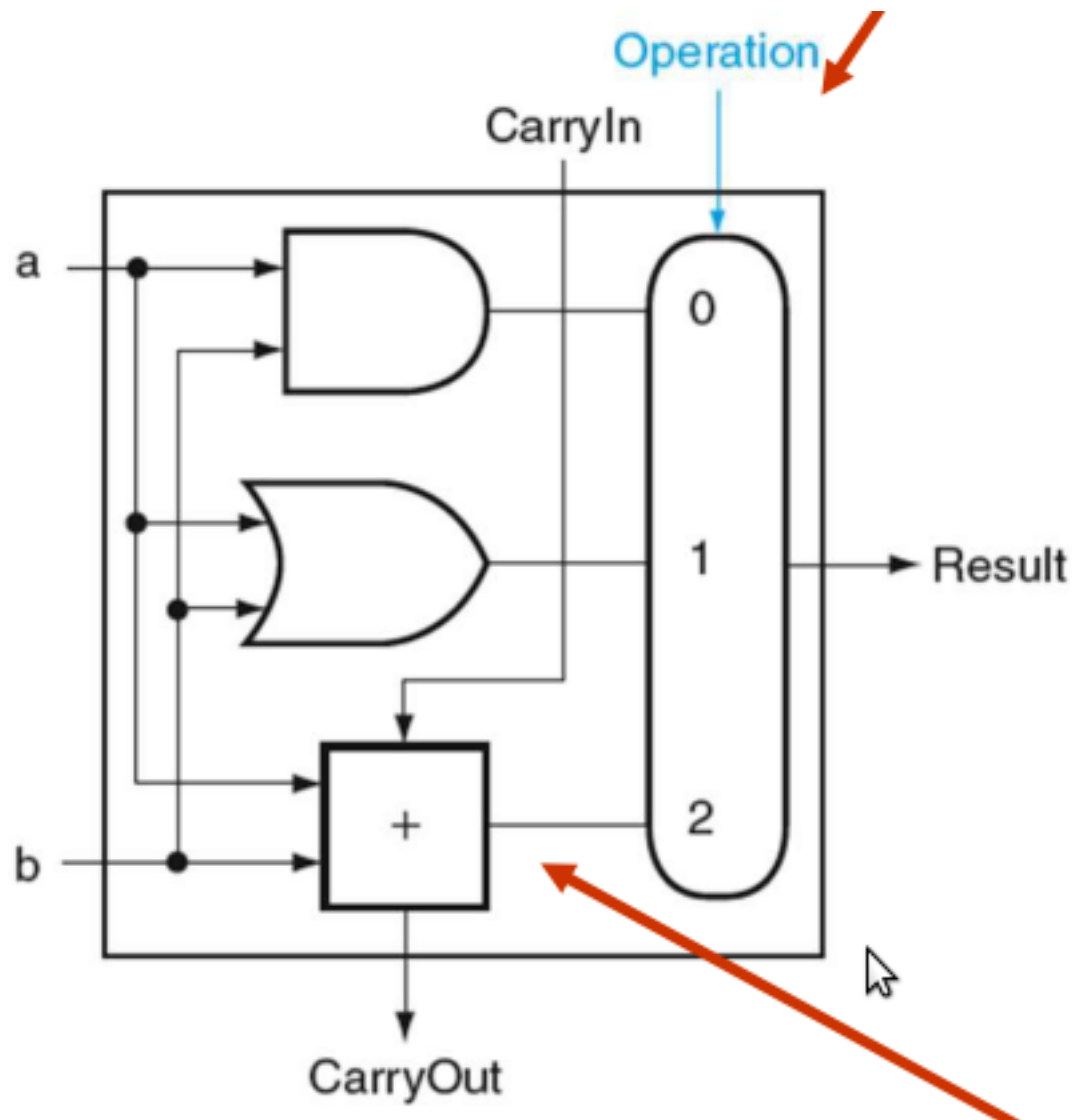


Figure 5: alt text

Questo circuito può fare 3 operazioni:

- AND
- OR
- Somma

Operation	Function
01	OR

Operation	Function
01	AND
10	SUM

Con il multiplexer selezioniamo l'operazione che vogliamo eseguire.(operation). Il circuito calcolerà sempre tutte le operazioni.

Per formare ALU a più bit, uniamo 32 ALU a 1 bit e ogni bit in ingresso lo assegnamo ad un'ALU.

Sottrazione

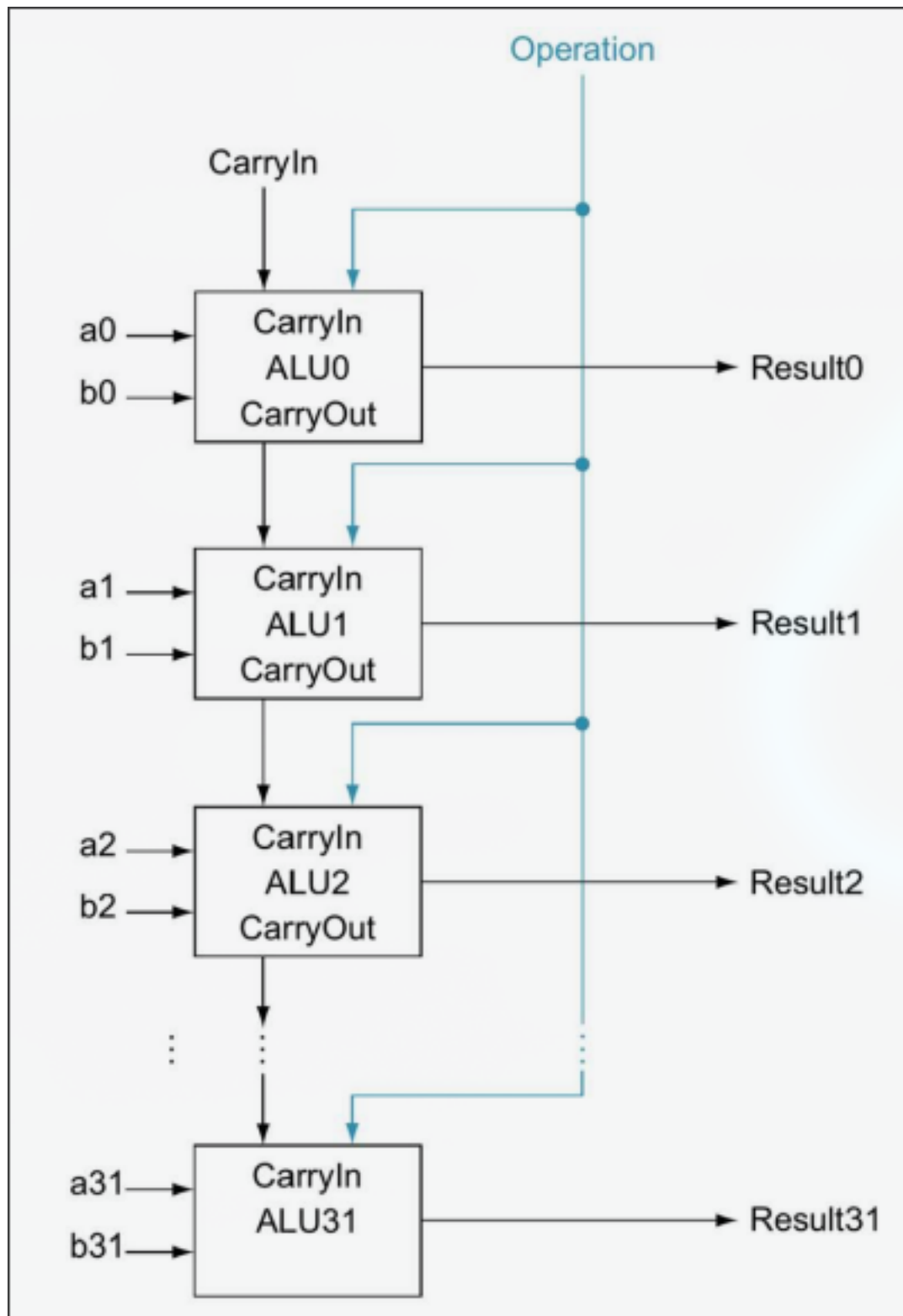
Per implementare la sottrazione $a - b$:

- Invertiamo il numero b
- Impostiamo il carryIn a 1 per il bit meno significativo attraverso un multiplexer che imposta 1 il carryIn se il bit più significativo è negato. Questo multiplexer sarà controllato da un ingresso aggiuntivo detto BInvert

Cioè convertiamo b in complemento a due.

ALU RISC-V a 32 bit

Una ALU a 32 bit è composta da 32 ALU ad un bit, dove il carryOut della ALU 1 è il carryIn della ALU 2 e così via.



NOR

Per implementare il NOR possiamo aggiungere un NOT all'inizio dell'ingresso A e controllarlo attraverso un ingresso aggiuntivo detto Ainvert. Inoltre tenendo Binvert dall'esempio prima, riusciamo anche a negare B all'occorrenza grazie a Binvert.

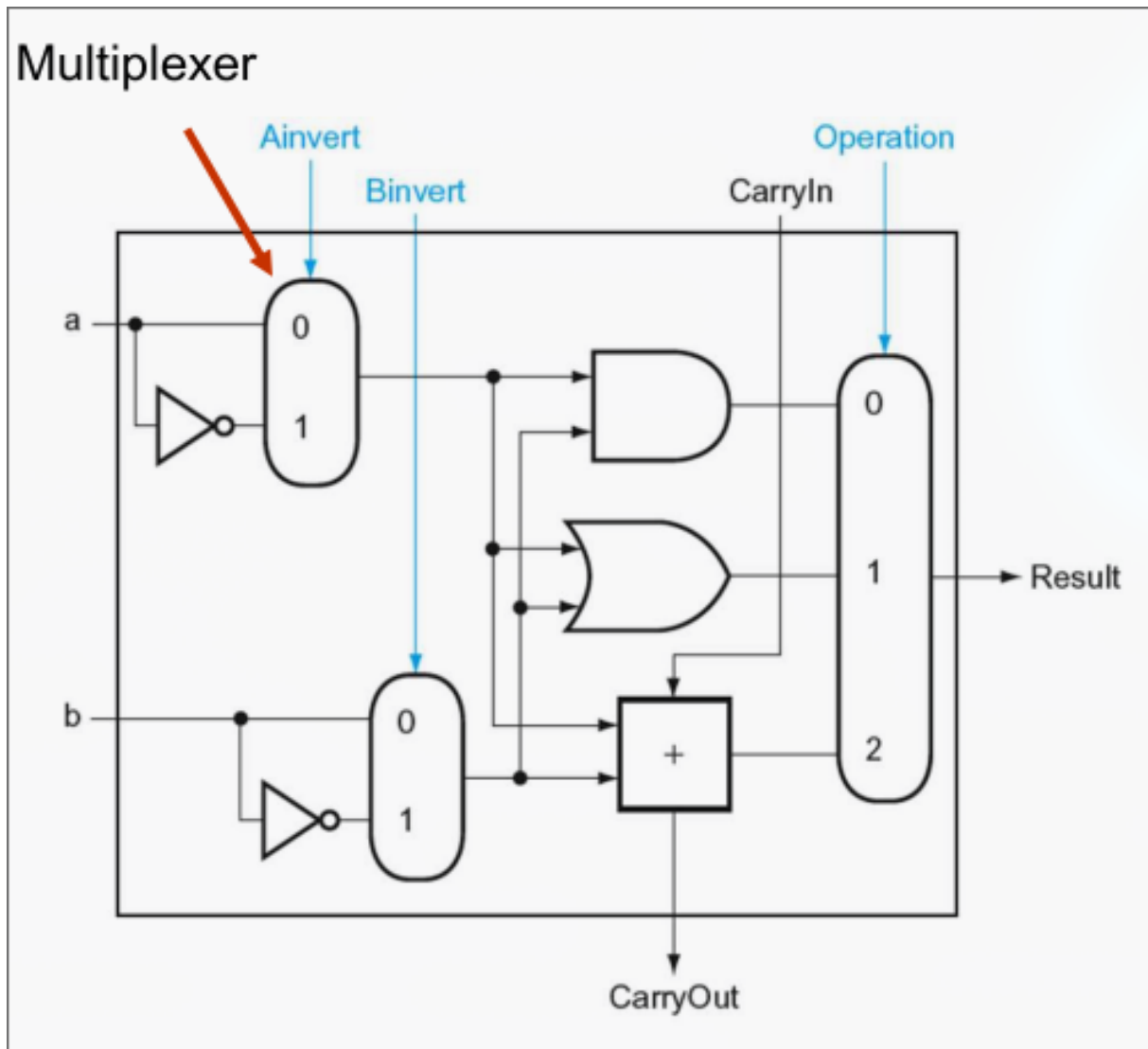


Figure 6: alt

Set on Less Than(SLT)

Per implementare SLT in un ALU ad un bit, usiamo la sottrazione ($a-b$) che è minore di 0 se $a < b$. Prendo poi questo risultato e verifico se il bit più significativo vale 1.

Per aggiungerlo nella ALU dei bit meno significativi dobbiamo fare i seguenti passi:

- Aggiungere un codice operativo 11 associato a LESS nel multiplexer
- Aggiungere un ingresso aggiuntivo LESS associato all'ultima uscita del multiplexer

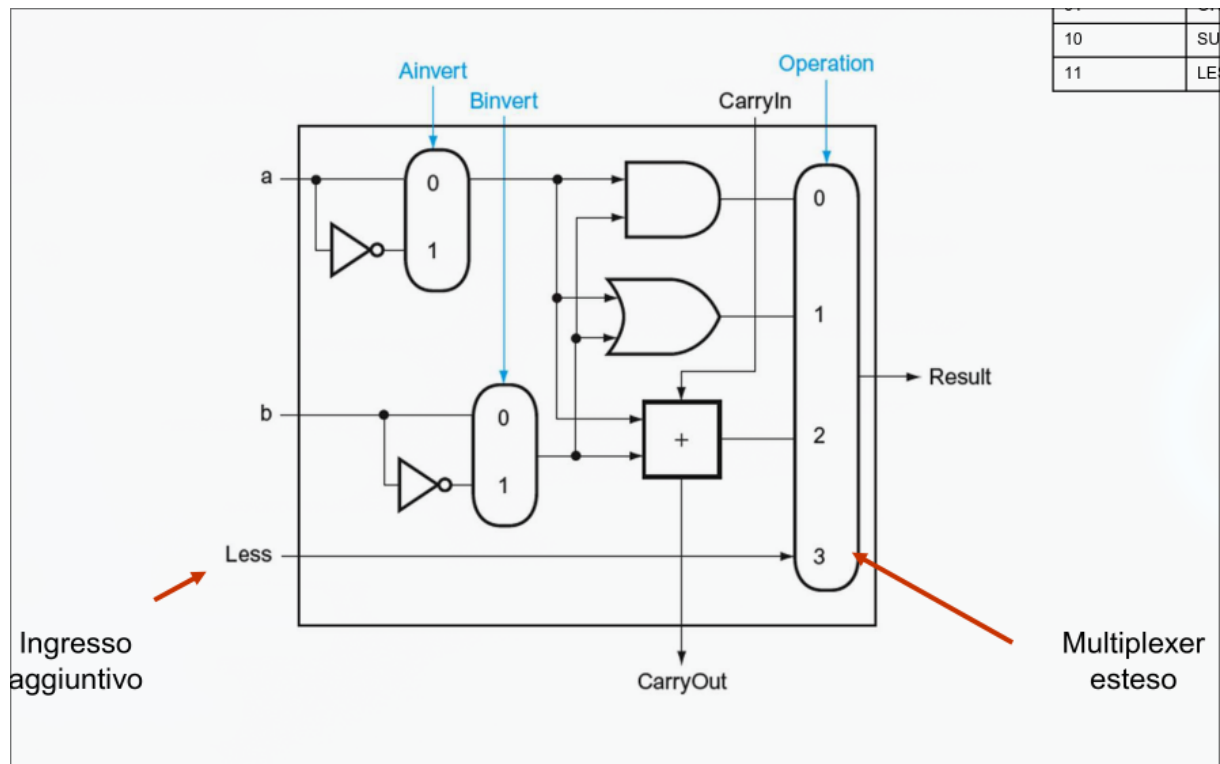


Figure 7: alt

Nella ALU del bit più significativo(32 esimo) dobbiamo controllare 1 o 0:

- Sarà uguale alle altre
- Dovrà controllare l'overflow attraverso una rete logica a parte. Questa rete avrà due uscite:
 - Overflow: 1 se c'è overflow o 0 se non c'è
 - Set: è uguale all'uscita del sommatore dell'ultimo bit. 1 se negativo, 0 se positivo.

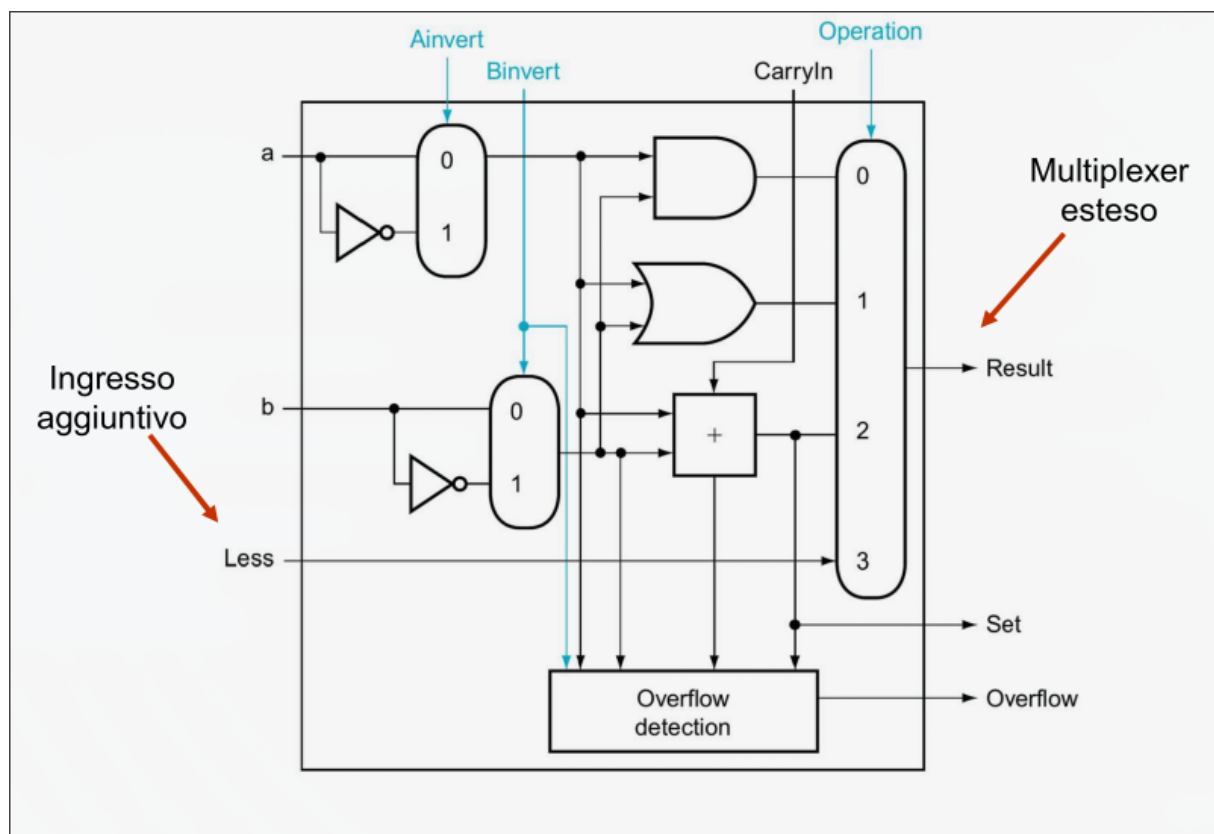


Figure 8: alt

A questo punto per implementare SLT il set sarà l'ingresso del bit meno significativo. Tutte le altre ALU avrà less a 0 e quindi si formerà il numero 0000...001

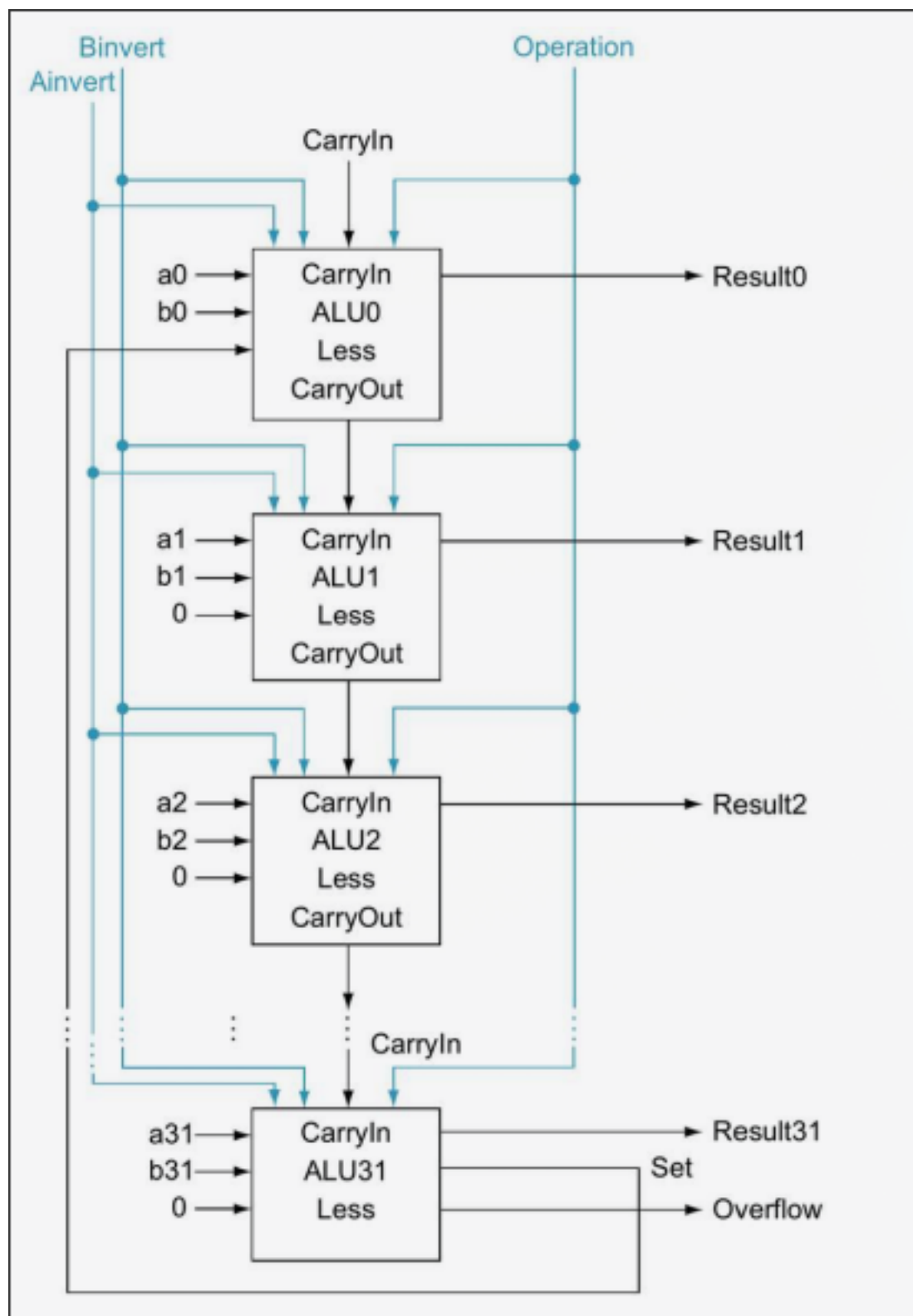


Figure 9: alt

Overflow detection

Dalla teoria del complemento a due, si ha overflow se CarryIn e CarryOut sono discordi nel bit più significativo. Quindi facciamo lo XOR tra questi due ingressi e questa sarà l'uscita di overflow.

Operation	Function	BIinvert	CarryIn0
01	OR	0	0
00	AND	0	0
10	ADD	0	0
10	SUB	1	1
11	SLT	1	1

Note: - CarryIn0 è il CarryIn al bit meno significativo - Per il SUB si usa l'addizionatore ma si usa il compl. a due

Notiamo che per implementare queste operazioni $\text{CarryIn0} = \text{BIinvert}$, quindi possiamo semplificare la rete creando un solo ingresso detto Bnegate.

Branch if Equal (beq)

Per implementare questo salto, si fa $a-b$ e si verifica se il risultato è 0.

Per fare ciò mettiamo in OR tutti i bit e poi lo neghiamo. In questo modo se c'è anche solo un bit ad uno, l'OR viene 1, negato 0, altrimenti viene 0, negato 1.

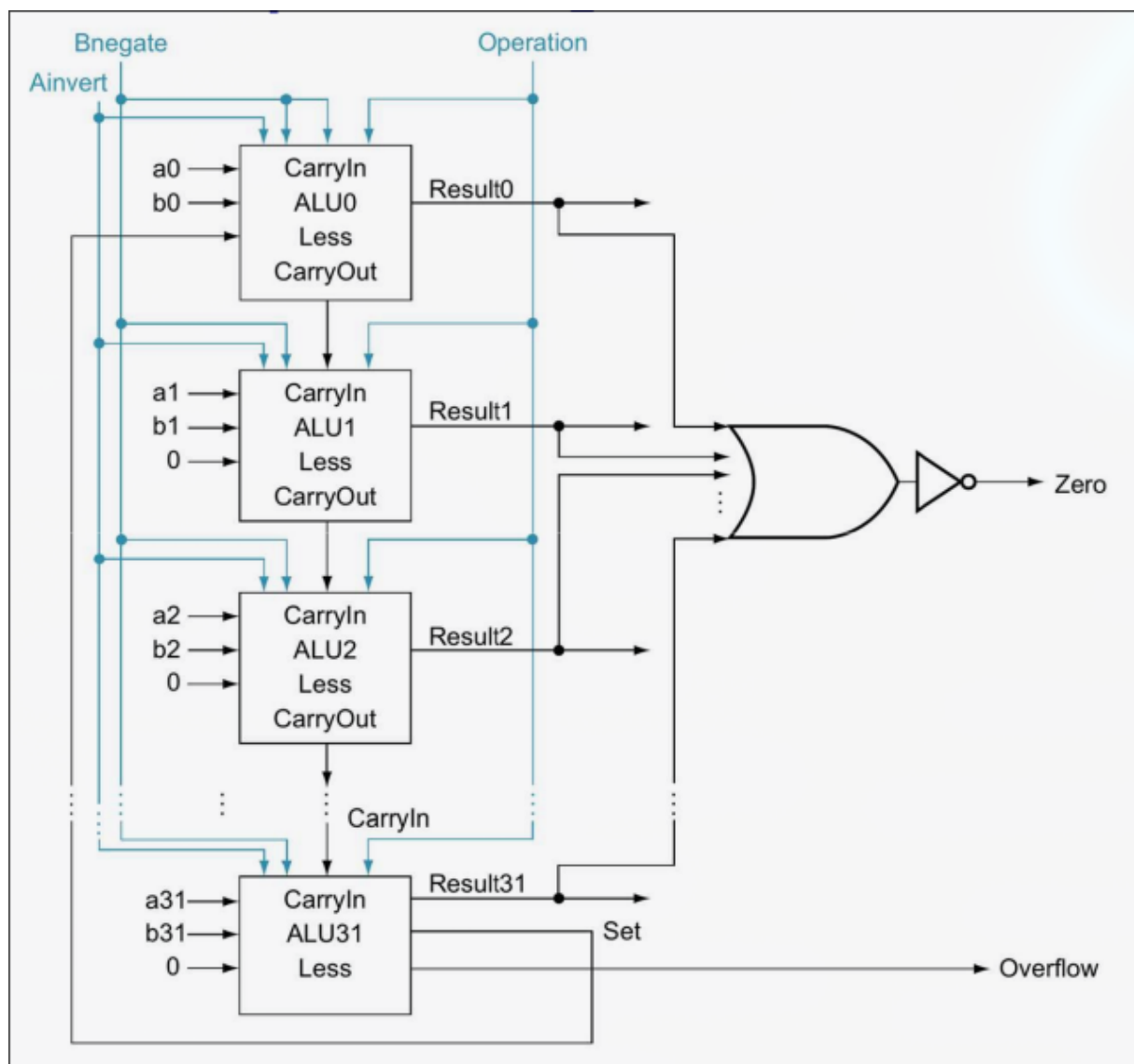


Figure 10: alt

Relazionare Opcode ed ingressi

Se rappresentiamo l'opcode su 4 bit:

Operation	Function
0000	OR
0001	AND

Operation	Function
0010	ADD
0110	SUB
0111	SLT
1100	NOR

AlInvert e BInvert verranno impostati di conseguenza in base all'opcode. Infatti possiamo notare che c'è una relazione tra opcode e AlInvert-BInvert:

Operation	Function	AlInvert	BInvert
0000	OR	0	0
0001	AND	0	0
0010	ADD	0	0
0110	SUB	0	1
0111	SLT	0	1
1100	NOR	1	1

L'ALU ora potrà essere vista come un solo blocco:

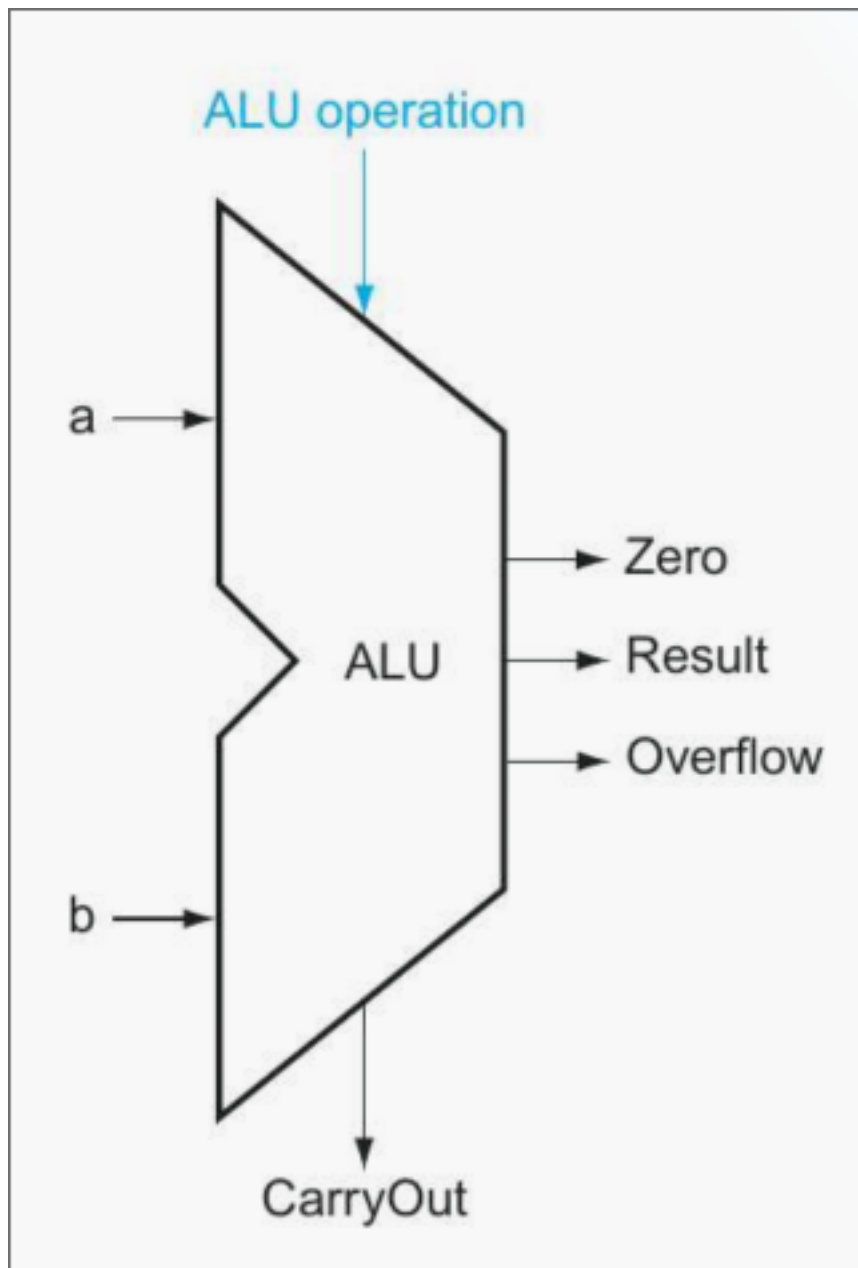


Figure 11: alt

Leggi delle algebre di Boole

- Associatività: $(x + y) + z = x + (y + z)$ e $(x \cdot y) \cdot z = x \cdot (y \cdot z)$
- Commutatività: $x + y = y + x$ e $x \cdot y = y \cdot x$
- Distributività: $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$ e $x + (y \cdot z) = (x + y) \cdot (x + z)$ e $x + (y \cdot z) = (x + y) \cdot (x + z)$

-
- Esistenza di elementi neutri: $x + 0 = x$ e $x \cdot 1 = x$
 - Esistenza di un inverso: $x + x' = 1$ e $x \cdot x' = 0$

L'inverso in un'algebra di Boole è univoco tale che:

$$x + y = 1 \text{ e } xy = 0 \text{ allora } y = x'$$

Inoltre soddisfa altre 6 proprietà:

- Idempotenza: $x + x = x$ e $xx = x$
- Legge del limite/dominanza: $x + 1 = 1$ e $x \cdot 0 = 0$
- Legge di assorbimento: $x + xy = x$ e $x(x + y) = x$
- Legge d'involuzione: $(x')' = x$
- Legge 0 e 1: $0' = 1$ e $1' = 0$
- Leggi di De Morgan per le algebre di Boole: $(x + y)' = x' \cdot y'$ e $(xy)' = x' + y'$

Forme canoniche

Formula normale disgiuntiva

Sommatoria di termini ciascuno dei quali è una produttoria di variabili in ingresso o da negazioni di variabili in ingresso

Formula normale congiuntiva

Produttoria di termini ciascuno dei quali è una sommatoria di variabili di ingresso o negazioni.

Formula di maggioranza

La formula di maggioranza restituisce 1 se la maggior parte dei termini è 1.

$$M = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Dalla tavola di verità alla funzione logica

Procediamo così:

- Scriviamo le formule che danno come output 1. Queste formule sono dette mintermini

-
- Disgiungiamole tra di loro

In questo modo otteniamo la formula disgiuntiva normale.